
Intelligent Network Intrusion Detection Using KNN and Feature Scaling for Enhanced Cyber Security Analysis.

Problem ID: 09
Team ID: PY-GT-58

Name : Sambhav Jaiswal(Team Leader)
Class Roll Number: 58
University Roll Number : 2028835
Section and Group: G(G-2)

Name : Pragun Khanna
Class Roll Number:54
University Roll Number : 2028798
Section and Group: G(G-2)

Name : Utsav Bajoria
Class Roll Number:73
University Roll Number : 2208896
Section and Group: G(G-2)

TABLE OF CONTENTS

1. Problem Description	3
1.1.Dataset Description.....	3
1.2.Flow diagram.....	4
2. Modules for Project Implementation.....	5
2.1. Modules Used	5-6
2.2. Platform Used.....	7-8
3. Machine Learning model/ Searching Technique used	8
4. Evaluation metrics used	9-10
5. Results and Visualizations	11-12
6. Conclusion and Future Scope	13
7. References.....	14

1. Problem Description

In the modern digital world, cybersecurity is a major concern due to the rapid increase in cyber attacks. Networks are constantly exposed to threats such as hacking, malware, and unauthorized access. Detecting these threats manually is difficult because of the large amount of data generated in network traffic.

This project aims to build a basic Intrusion Detection System (IDS) using Machine Learning techniques. The system will analyse network data based on features like Packet Size, Connection Duration, and Error Rate. These features help in identifying patterns that indicate whether the traffic is normal or suspicious. The K-Nearest Neighbours (KNN) algorithm will be used for classification. It is simple and suitable for beginners as it classifies data based on similarity. The Elbow Method will be used to find the optimal value of K, which improves the model's accuracy. This project helps first-year students understand how machine learning can be applied in real-world cybersecurity problems.

1.1 Dataset Description

The dataset used in this project is a structured network traffic dataset designed for Intelligent Network Intrusion Detection. It contains multiple features representing different aspects of network behavior, such as packet size, connection duration, error rate, protocol type, source and destination details, and other security-related parameters. These features help in analyzing traffic patterns and identifying suspicious or abnormal activities within the network.

The dataset includes both normal and malicious network traffic records, where each entry is labeled for binary classification:

Label = 0 → Normal Traffic

Label = 1 → Intrusion / Malicious Traffic

This dataset was used to perform Exploratory Data Analysis (EDA), data cleaning, preprocessing, feature engineering, and machine learning model implementation. Its purpose is to train and evaluate classification algorithms such as KNN, Logistic Regression, SVM, Decision Tree, and Random Forest to accurately distinguish between safe network activity and potential cyber threats. The dataset provides a practical foundation for understanding cybersecurity threats and building an efficient intrusion detection system.

1.2 Flow Diagram



Fig1: Flow Diagram

2. Modules for Project Implementation

2.1 Modules Used(mention all the inbuilt/ custom modules used and the purpose)

The following inbuilt Python modules and machine learning libraries were used during the implementation of the Intelligent Network Intrusion Detection System project:

1. Pandas

Used for data loading, data manipulation, cleaning, handling missing values, and working with CSV files in tabular format.

2. NumPy

Used for numerical operations, mathematical calculations, array processing, and statistical computations.

3. Matplotlib

Used for basic data visualization such as histograms, line plots, and graphical representation of data patterns.

4. Seaborn

Used for advanced visualization including heatmaps, boxplots, correlation matrices, and confusion matrix visualization.

5. Scikit-learn (sklearn)

Used for machine learning model implementation, preprocessing, training, testing, and evaluation.

Major sklearn modules used:

a) LabelEncoder

Used for converting categorical text data into numerical form.

b) StandardScaler

Used for feature scaling to normalize numerical data for algorithms like KNN and SVM.

c) train_test_split

Used to divide the dataset into training and testing sets.

d) KNeighborsClassifier

Used to implement the K-Nearest Neighbors algorithm.

e) LogisticRegression

Used for binary classification and baseline model comparison.

f) SVC (Support Vector Classifier)

Used for Support Vector Machine model implementation.

g) DecisionTreeClassifier

Used for Decision Tree classification.

h) RandomForestClassifier

Used for Random Forest classification.

i) accuracy_score

Used to calculate model accuracy.

j) classification_report

Used to generate Precision, Recall, F1-Score, and Support.

k) confusion_matrix

Used to evaluate prediction performance using TP, TN, FP, and FN.

2.2 Platform Used: (mention all the platforms with the reason of using it)

1.Jupyter Notebook:

Used for coding, step-by-step execution, data analysis, and visualization.

2.Python:

Used as the main programming language for preprocessing, machine learning, and evaluation.

3.Excel / CSV:

Used for dataset creation, storage, and management.

4.Windows:

Used as the operating system for running all project tools and software.

3. Machine Learning Model/ Searching Technique Used(Describe each ML model/ searching technique used)

1. K-Nearest Neighbors (KNN)

KNN is a supervised machine learning classification algorithm that classifies data based on the similarity of nearest neighboring data points. It identifies the K closest samples and assigns the majority class. In this project, KNN is the primary model used for detecting whether network traffic is normal or intrusive.

2. Elbow Method

The Elbow Method is a searching and optimization technique used to determine the optimal value of K for the KNN algorithm. It analyzes error rates for multiple K values and helps select the most suitable number of neighbors for better model performance.

3. Logistic Regression

Logistic Regression is a supervised classification model used for binary classification. It predicts whether network activity is normal or malicious based on probability scores and serves as a baseline model for comparison.

4. Support Vector Machine (SVM)

SVM is a supervised learning algorithm that classifies data by finding the best hyperplane separating different classes. It is effective for identifying complex intrusion patterns in network traffic.

5. Decision Tree

Decision Tree is a rule-based supervised learning model that classifies data through branching decisions based on feature conditions. It helps in understanding feature importance and intrusion logic clearly.

6. Random Forest

Random Forest is an ensemble learning model that combines multiple decision trees to improve prediction accuracy and reduce overfitting. It is used for robust and reliable intrusion detection performance.

4. Evaluation metrics used (Describe each evaluation metric used along with formulae)

1. Accuracy

Accuracy measures the overall correctness of the model by calculating the ratio of correctly predicted instances to total instances.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

TP = True Positive

TN = True Negative

FP = False Positive

FN = False Negative

2. Precision

Precision measures how many predicted positive intrusion cases were actually correct.

$$Precision = \frac{TP}{TP + FP}$$

Purpose:

Used to reduce false alarms.

3. Recall

Recall measures how many actual intrusion cases were correctly detected by the model.

$$Recall = \frac{TP}{TP + FN}$$

Purpose:

Used to minimize missed attacks.

4. F1-Score

F1-Score is the harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Purpose:

Provides balanced model performance.

5. Confusion Matrix

Confusion Matrix is a performance evaluation table that shows:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

Purpose:

Helps analyze correct predictions, false alarms, and missed intrusions in detail.

6. Classification Report

Classification Report provides a complete summary of:

- Precision
- Recall
- F1-Score

- Support

Purpose:

Used for detailed performance comparison of all machine learning models.

5. Results and Visualizations

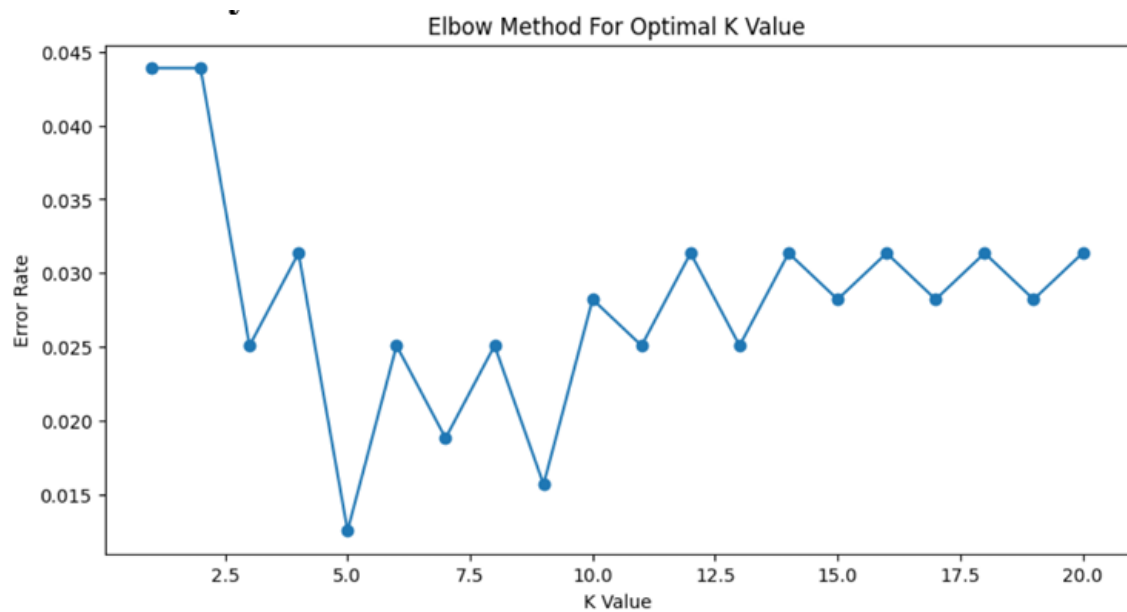


Fig.1: Showing graph for elbow method showing optimal K value

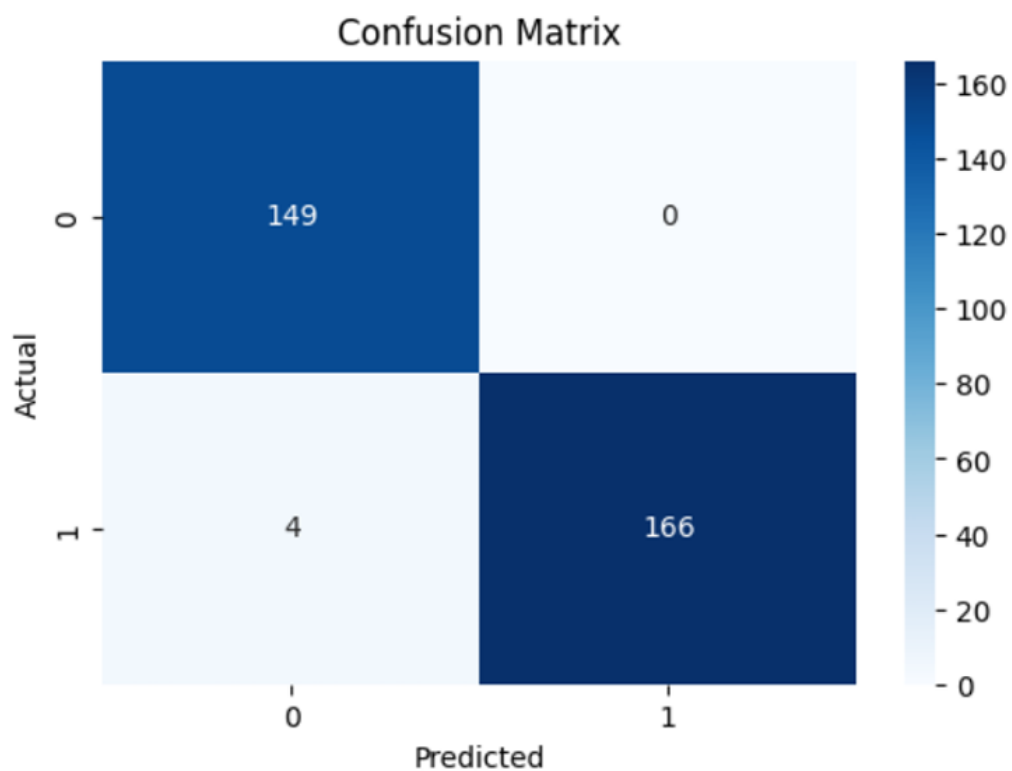


Fig 2. Graph of Confusion Matrix showing the actual value and predicted value.

LOGISTIC REGRESSION
Accuracy: 99.05956112852664 %

Classification Report:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	149
1	1.00	0.98	0.99	170
accuracy			0.99	319
macro avg	0.99	0.99	0.99	319
weighted avg	0.99	0.99	0.99	319

Confusion Matrix:
[[149 0]
[3 167]]

SUPPORT VECTOR MACHINE
Accuracy: 99.37304075235109 %

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	149
1	1.00	0.99	0.99	170
accuracy			0.99	319
macro avg	0.99	0.99	0.99	319
weighted avg	0.99	0.99	0.99	319

Confusion Matrix:
[[149 0]
[2 168]]

Fig 3

Fig.3

6. Conclusion and Future Scope:

Conclusion

The Intelligent Network Intrusion Detection System project successfully applied data analysis, preprocessing, and multiple machine learning algorithms to detect normal and malicious network traffic. Through Exploratory Data Analysis (EDA), data cleaning, feature engineering, and model implementation, important traffic patterns and intrusion-related behaviors were identified. Algorithms such as KNN, Logistic Regression, SVM, Decision Tree, and Random Forest were used to classify network activities and compare performance using evaluation metrics like Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report. The project demonstrates that machine learning can effectively enhance cybersecurity by improving intrusion detection accuracy and identifying suspicious activities in network environments.

Future Scope

In the future, this project can be improved by using larger real-world cybersecurity datasets and implementing advanced machine learning or deep learning models such as Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), or Deep Learning-based Intrusion Detection Systems. Real-time network monitoring, cloud security integration, and anomaly-based detection can also be added to increase practical applicability. Further optimization through hyperparameter tuning, feature selection, and deployment as a real-time security application can make the system more accurate, scalable, and suitable for modern cybersecurity challenges.

7. References:

1. Scikit-learn Documentation

Used for machine learning algorithms such as KNN, Logistic Regression, SVM, Decision Tree, Random Forest, and evaluation metrics.

2. Pandas Documentation

Used for data loading, preprocessing, cleaning, and dataset manipulation.

3. NumPy Documentation

Used for numerical operations and mathematical computations.

4. Matplotlib Documentation

Used for data visualization including graphs and plots.

5. Seaborn Documentation

Used for advanced visualizations such as heatmaps, boxplots, and correlation matrices.

6. Jupyter Notebook

Used as the development platform for coding, implementation, and analysis.

7. Python Official Documentation

Used for understanding Python syntax, functions, and programming concepts.

8. Research Articles / Cybersecurity Learning Resources

Used for understanding intrusion detection systems, network security concepts, and machine learning applications in cybersecurity.